以下是基于您提供的课件（文档1：Chapter 14: Concurrency Control）和真题（文档2）的详细总结与讲解。我会严格遵循课件内容，翻译英文术语，全面覆盖知识点，并举例说明。回复结构包括：知识点总结、举例讲解、例题巩固。课件中的图片标签我会在相关位置嵌入，以增强理解（图片来自课件，如无特别说明，页码参考课件编号，例如第18.8页）。

数据库并发控制详解

一、引言

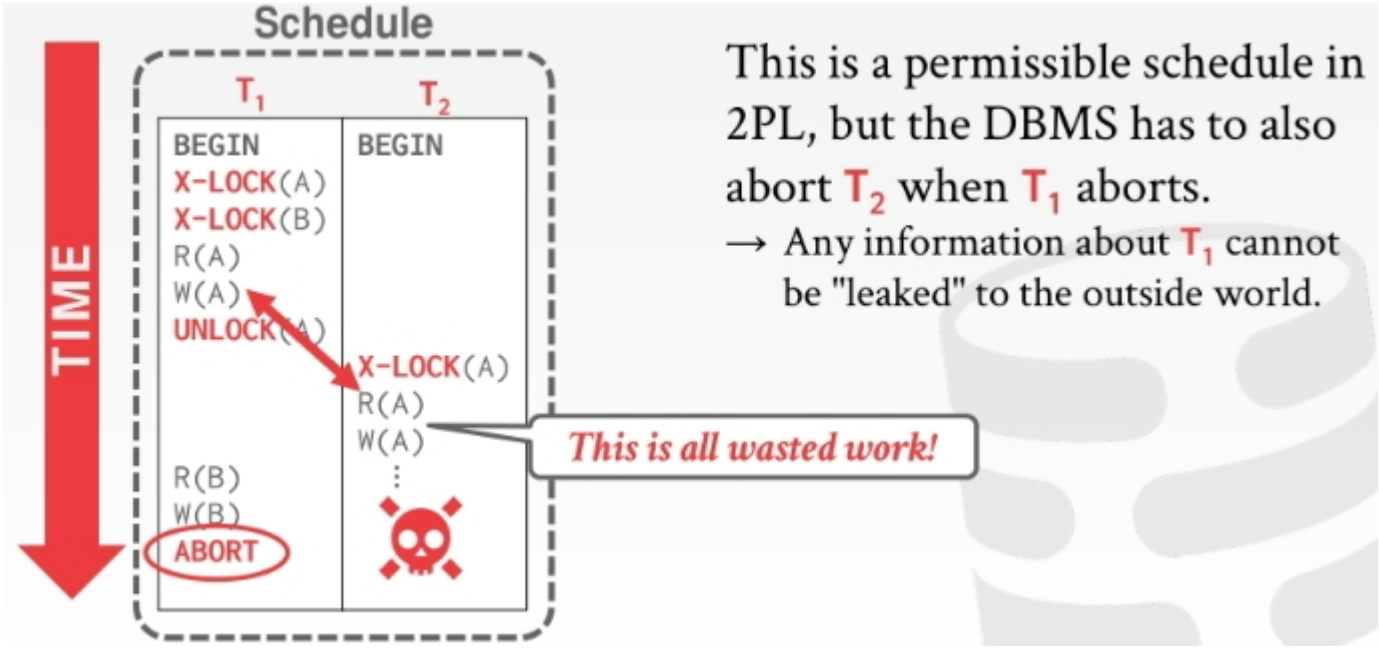
并发控制是数据库系统的核心机制，用于管理多个事务同时访问数据时的冲突，确保 **可串行化**（即所有并发执行的结果等价于某个串行执行）。课件以锁、时间戳和多版本方案为主，我将按顺序总结。

二、锁基于协议（Lock-Based Protocols）

锁是控制并发访问的基本机制，课件定义了两种锁模式：

- **共享锁（S锁）**：用于读操作，多个事务可同时持有。
- **排他锁（X锁）**：用于写操作，只允许一个事务持有。

锁兼容矩阵如下（课件第18.5页）：



- S锁与S锁兼容，但X锁与任何锁都不兼容。

例子：假设事务T1和T2同时访问数据项A。

- 如果T1持有S锁读A，T2可以申请S锁读A（兼容）。
- 如果T1持有X锁写A，T2申请任何锁都会被阻塞。
- 课件第18.6页的例子：事务T2执行锁操作，但单纯锁不足以保证可串行化，需要协议约束。

说明：锁基于协议要求事务在访问数据前先申请锁，但如果不加约束，可能导致非可串行化调度。例如，事务提前释放锁可能导致"脏读"。

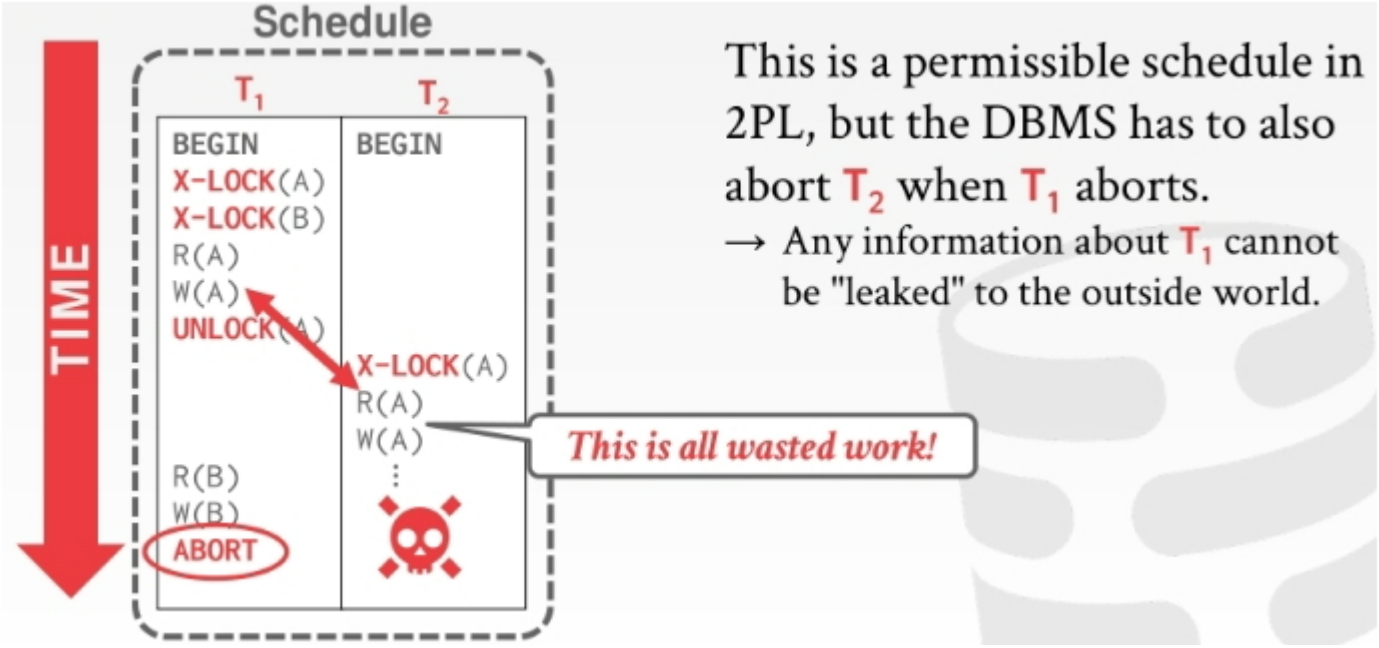
三、两阶段封锁协议（Two-Phase Locking Protocol, 2PL）

这是保证冲突可串行化的关键协议（课件第18.8页重点）。

- **生长阶段**：事务只能获取锁，不能释放锁。
- **收缩阶段**：事务只能释放锁，不能获取锁。

例子 ：课件第18.9页的事务T1。

- 符合2PL的版本：T1先锁S(B)和X(A)，执行操作后统一释放锁。
- 不符合2PL的版本：T1在写A前释放了B的锁，可能破坏可串行化。



扩展 ：

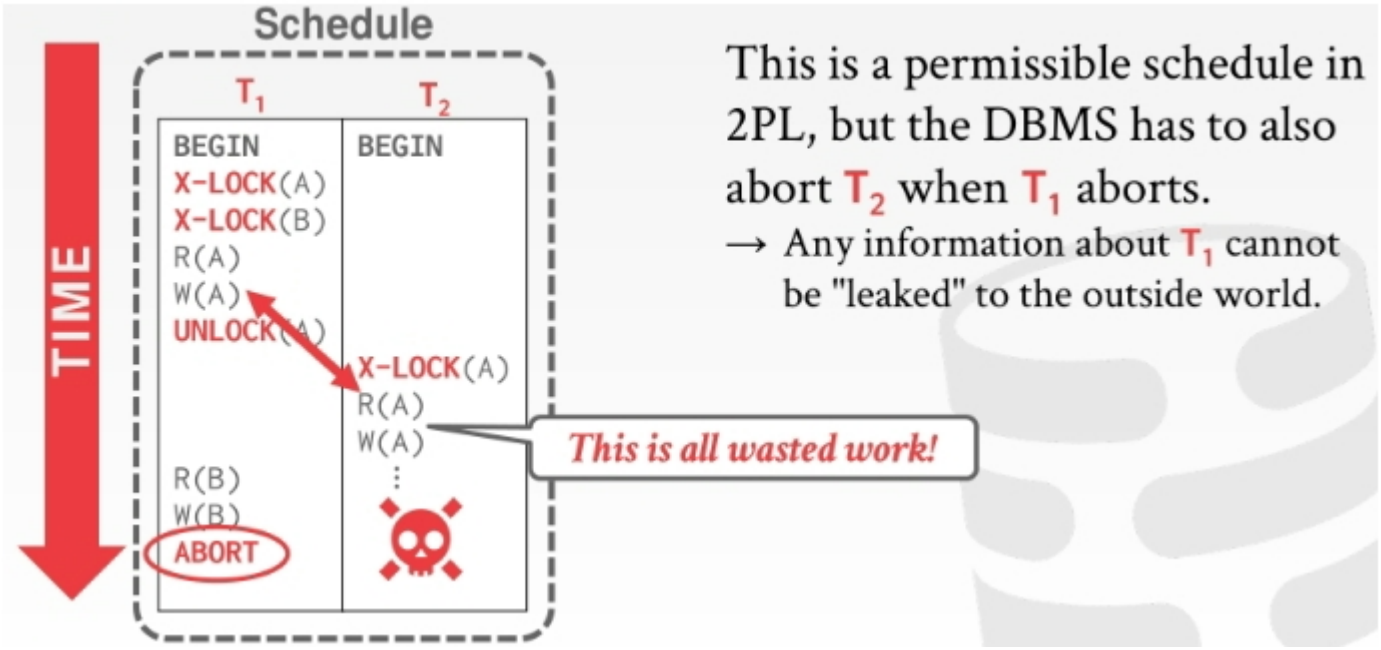
- **严格2PL** ：事务持有所有X锁直到提交，避免级联回滚（课件第18.11页）。
- **严谨2PL** ：事务持有所有锁直到提交，确保提交顺序即串行顺序。

为什么2PL保证可串行化？ 事务的锁点（获取最后一个锁的时间）决定了串行顺序。

四、死锁处理（Deadlock Handling）

死锁是多个事务相互等待锁的场景（课件第18.18页）。

例子 ：事务T3锁A，T4锁B；然后T3申请锁B，T4申请锁A，形成循环等待。



处理方案 ：

- **预防** ：如等待-死亡（非抢占）或伤害-等待（抢占）协议（课件第18.23页）。

- **检测** ：使用等待图（Wait-for Graph），有环则表示死锁（课件第18.25页）。
- **超时机制** ：简单但可能误判。

五、时间戳基于协议（Timestamp-Based Protocols）

每个事务有唯一时间戳，调度按时间戳顺序执行（课件第18.46页）。

- **时间戳排序协议（TSO）** ：比较事务时间戳与数据项的W-timestamp（最后写时间戳）和R-timestamp（最后读时间戳）。
- **规则** ：
- 读操作：如果事务时间戳 < W-timestamp，则回滚（读旧值）。
- 写操作：如果事务时间戳 < R-timestamp 或 < W-timestamp，则回滚（写冲突）。

例子 ：课件第18.50-18.56页的调度示例。

- 事务T25（时间戳25）和T26（时间戳26）访问数据A和B。如果T26写A后T25读A，且T25时间戳小，则T25回滚。

Thomas写规则 ：忽略过时写操作，提高并发性（课件第18.59页）。

六、多版本方案（Multiversion Schemes）

维护数据项的多个版本，读操作返回事务开始时的最新版本（课件第18.68页）。

- **多版本时间戳排序（MVTO）** ：每个版本有W-timestamp和R-timestamp。
- **多版本2PL** ：读事务用时间戳，写事务用锁。

例子 ：课件第18.73页，数据A有版本A0（值123）和A1（值456）。事务T1（时间戳1）读A返回A0，T2（时间戳2）写A创建A1。

七、快照隔离（Snapshot Isolation, SI）

读事务看到数据库某一时刻的快照，写事务遵循"先提交者获胜"规则（课件第18.79页）。

- **优点** ：读不阻塞写，避免脏读、丢失更新。
- **问题** ：可能违反可串行化，如写偏斜（课件第18.84页）。

例子 ：事务T1和T2同时读A和B，然后分别写A和B。串行执行应顺序更新，但SI下可能两者都成功，导致数据不一致。

八、弱一致性级别

如度二一致性、游标稳定性，用于提高并发但牺牲串行化（课件第18.90页）。SQL中设置隔离级别（如READ COMMITTED）。

九、例题巩固

基于课件和真题（文档2），我编写以下例题帮助理解。真题中事务特性（ACID）、锁协议是重点。

例题1：选择题（类似真题一）

问题 ：两阶段封锁协议的目的是什么？

A. 提高查询速度

B. 保证冲突可串行化

C. 减少死锁

D. 简化锁管理

答案 ：B

解析 ：课件第18.8页强调，2PL通过生长和收缩阶段限制锁操作，确保冲突可串行化。

例题2：简答题（类似真题三）

问题 ：简述事务的ACID特性，并举例说明隔离性。

答案 ：

- Atomicity（原子性）：事务操作全做或全不做，如转账要么成功要么失败。
- Consistency（一致性）：事务使数据库从一致状态到另一一致状态，如余额不为负。
- Isolation（隔离性）：并发事务互不干扰。例如，事务T1读A时，T2写A会被阻塞，避免脏读。
- Durability（持久性）：提交后修改永久保存。

参考 ：真题简答题第2题，课件第18.58页提到时间戳协议保证串行化但可能不保证隔离性。

例题3：综合题（类似真题四）

问题 ：给定事务序列：

T1: lock-X(A); write(A); unlock(A);

T2: lock-S(A); read(A); unlock(A); lock-X(B); write(B); unlock(B);

判断是否满足2PL？如果不满足，如何修改？

答案 ：

- 分析：T2在读A后释放锁，然后申请锁B，这违反2PL的生长阶段（不能释放锁后再获取）。
- 修改：T2应持所有锁直到操作结束，如lock-S(A); read(A); lock-X(B); write(B); unlock(A); unlock(B)。

参考 ：课件第18.9页的2PL例子。

例题4：死锁检测

问题 ：事务T1锁A，T2锁B，T1申请B，T2申请A。画出等待图，并说明如何处理。

答案 ：

- 等待图：T1 → T2（等B），T2 → T1（等A），有环。
- 处理：系统检测到死锁后，回滚一个事务（如T1），释放其锁。

参考 ：课件第18.25页的等待图示例。

十、总结

- **重点** ：锁协议（尤其是2PL）和时间戳协议是并发控制的核心，真题中事务特性和锁机制频繁出现。
- **学习建议** ：结合课件的例子理解调度，多做例题巩固。课件图片有助于可视化冲突（如第18.8页的2PL示例）。
- **注意** ：课件以英文为主，但概念通用，翻译后更易理解。如有细节问题，可参考具体页码。

通过以上总结和例题，您应该能巩固并发控制知识。如果有特定概念需要深入，请告知！